

Code Smells and Detection Techniques: A Survey

Rana S. Menshawy
Computer and System Department,
Faculty of Engineering
Ain Shams University
Cairo, Egypt
G18092628@eng.asu.edu.eg

Ahmed H. Yousef
School of Information Technology and
Computer Science (ITCS), Nile
University
Cairo, Egypt
ahassan@NU.edu.eg

Ashraf Salem
Computer and System Department,
Faculty of Engineering
Ain Shams University
Cairo, Egypt
ashraf.salem@eng.asu.edu.eg

Abstract— *Design and code smells are characteristics in the software source code that might indicate a deeper design problem. Code smells can lead to costly maintenance and quality problems, to remove these code smells, the software engineers should follow the best practices, which are the set of correct techniques which improve the software quality. Refactoring is an adequate technique to fix code smells, software refactoring modifies the internal code structure without changing its functionality and suggests the best redesign changes to be performed. Developers who apply correct refactoring sequences to remove code smells, improve the software maintenance and development time significantly. Many tools have been created to automatically or semi-automatically detect code smells and refactor them. This study discusses the code smells, detection techniques, detection and refactoring tools, in addition to observing the challenges and suggesting enhancements for better performance.*

Keywords—Code smells, Software Refactoring, detection techniques, Survey.

I. INTRODUCTION

Code smells refer to **any sign that possibly can negatively affect the software process in a deeper way affecting the quality and maintainability of software** [1]. Smells occurrence leads to a difficulty in modifying or understanding the code, implementing new features and fixing program issues become complex and time-consuming, making the software hard to evolve and maintain. Adding new requirements will be a difficult mission due to the code complexity and the design flaws [2]. Smells are subjective in their definitions and categorizations, besides that not all smells are considered bugs, but they may affect the software design negatively in the future.

The term “Refactoring” was introduced earlier by Opdyke and Fowler to remove code smells [1], software refactoring is a well-known technique that is used by developers to remove code smells and consequently improve the design and the evolution of any system. Refactoring aims to change the internal code structure without changing the external behavior improving the code flexibility and maintainability [3]. Refactoring is done by applying a sequence of small transformations preserving the same functionality [4]. Although software refactoring is a time-consuming process that does not reflect immediate benefits like adding new features or fixing bugs, it allows any developer to read, maintain, understand and easily extend any software system [5].

There are many challenges to apply the proper refactoring, such as how to detect a code smell, what is the refactoring technique should be followed and the gain of applying it [2], selecting a refactoring technique depends on the number of removed smells, the number of modified elements and how the code will be maintained [6]. Applying a correct sequence of refactoring techniques is very crucial because **any wrong or incomplete refactoring can result in creating new smells and flaws in the code** [7].

Many developers are not aware of code smells [8], in addition to that identification of smells requires training and experience [9]. Automatic or semi-automatic detection and refactoring tools that support the identification of code smells and refactoring them, help the developers to save their effort and time. **Most of the modern integrated development environments (IDEs) such as Visual Studio and Eclipse support software refactoring** [10].

Previous survey studies discussed the challenges of code smells by analyzing the code smells definitions, detection techniques, detection and refactoring tools [2, 3, 9, 10, 11, 41]. Some studies discussed the awareness of developers about code smells, how the developers perceived code smells and the motivation for removing the smells [6, 8, 16, 42]. Other studies discussed the refactoring activities and limitations [5, 7, 12, 40], clone management and activities are discussed in the study [15]. Some studies discussed the approaches and algorithms used in the detection and refactoring tools [43, 44, 45, 46].

This study intends to complement such previous investigations by discussing and identifying code smells, detection approaches, detection and refactoring tools. The paper is organized as follows: Section 2 describes the paper methodology. Section 3 explores the code smells. Section 4 discusses the detection approaches and algorithms and explored the most cited detection and refactoring tools. Section 5 explores our findings and concludes the paper.

II. METHODOLOGY

In this section, the objective of this systematic literature review is discussed, and the criteria of the included relevant papers are identified.

The search process is involved in Scopus digital library (<https://www.scopus.com/>), by applying a searching query: (“Code Smells” and “Refactoring” and “Survey”) or (“Code Smells” and “detection techniques” and “Review”) with date range between 2006 and 2020. Scopus is chosen among other databases such as Web of Science and Google Scholar [39].

This is because Web of science includes journals and conferences less than Scopus as it covers only journals of influence, Google scholar indexed any document with a seemingly academic structure including more citations than Web of Science and Scopus across all subject areas in a significant way because it covers a lot of non-reviewed content and lower quality citing documents.

This study is based on 19 previous studies, the objective is to highlight the prior contributions in the code smells and the refactoring fields, the purpose of this study literature review is to answer the following questions:

R1: What are the code smells discussed in the prior studies?

R2: What are the detection approaches and algorithms?

R3: What are the common tools used for code smell detection and refactoring?

The criteria for inclusion include:

- Papers describe the evaluation/ development/ usage of detection techniques, detection, or refactoring tools.
- Only full papers are included.
- Papers that published in journals or conferences or workshops
- Works published between 2006 and 2020 including the recent papers.
- Papers published with no author name are not included.

III. CODE SMELLS

Twenty-two code smells were defined earlier by Fowler et al. (1999) [1], then Wake (2003) [4] add other seven smells such as Complicated Boolean Expressions, Dead Code, Special Case, Null Check, Inconsistent Names, Uncommunicative Names and Names with Embedded Types. Kerievsky (2004) [13] add other design smells such as Oddball Solution, Indecent Exposure, Solution Sprawl, Combinatorial Explosion and Conditional Complexity, after that Fowler and Beck (2019) [14] added other recent smells to the smells list such as Global Data, Loops, lazy elements, Mysterious name, Insider Trading and Mutable Data. Mantyla, (2003) [41, 44, 50] proposed categorization for the smells defined by fowler.

The twenty-two code smells proposed by fowler are the most cited ones in the previous papers. From the developers' point of view Duplicated Code, Long Method, and Accidental Complexity are the most important smells [16], because developers do not consider all smells to be harmful, only smells related to complexity and size are considered a threat to the software quality [8]. In Table I, the twenty-two smells and the suggested categories by Mantyla are presented.

TABLE I. CODE SMELLS

Code Smell	Category	References
Duplicated Code	Dispensables	[1, 2, 11, 15, 45]
Feature Envy	Couplers	[1, 2, 3, 7, 9, 11, 45, 46]
Long Method/ Long Function	Bloaters	[1, 2, 2, 3, 9, 11, 12, 45, 46]
Large Class	Bloaters	[1, 2, 9, 11, 45, 46]
Long Parameter List	Bloaters	[1, 2, 11, 45, 46]
Data Class	Dispensables	[1, 2, 3, 11, 45]

Code Smell	Category	References
Divergent Change	Change Preventers	[1, 2, 9, 11, 45]
Shotgun Surgery	Change Preventers	[1, 2, 9, 11, 45]
Data Clumps	Bloaters	[1, 2, 11, 45]
Primitive Obsession	Bloaters	[1, 2, 11, 45]
Switch Statements/ Repeated switches	Object-Orientation Abusers	[1, 2, 2, 11, 45]
Parallel Inheritance Hierarchies	Object-Orientation Abusers	[1, 2, 11, 45]
Lazy Class	Dispensables	[1, 2, 11, 45]
Speculative Generality	Dispensables	[1, 2, 11, 45]
Temporary Field	Object-Orientation Abusers	[1, 2, 11, 45]
Message Chains	Encapsulators	[1, 2, 11, 45]
Middle Man	Encapsulators	[1, 2, 11, 45]
Inappropriate Intimacy	Couplers	[1, 2, 11, 45]
Alternative Classes with Different Interfaces	Object-Orientation Abusers	[1, 2, 11, 45]
Incomplete Library Class	-	[1, 2, 11, 45]
Refused Bequest	Object-Orientation Abusers	[1, 2, 11, 45, 46]
Comments	-	[1, 2, 11, 45]

Fowler proposed more than seventy refactoring techniques in order to remove the code smells [1], difficulty in readability, re-usability, debugging and testing are the most important reasons which push developers to refactor their code [5]. **More than one technique can be used to refactor one code smell**, "Extract Method" is considered one of the most used refactoring techniques used by many smells [3]. code quality should be improved by applying proper refactoring, which can be measured by different metrics such as coupling and size, in addition to the test cases which could easily detect smells and bugs earlier in the software development and validate the software quality against code smells [42].

IV. DETECTION APPROACHES

There are many detection approaches such as manual approach, metrics-based, history-based, machine learning-based, software visualization, semi-automated and fully automated tools. The automatic tools approach is the most used approach to detect smells [12].

Manual approach [2, 11] detects code smells and design flaws according to the human perception by applying the detection guidelines, which is a time-consuming process in addition to human error, this approach is extremely difficult and ineffective in case of detecting smells in large scale software applications.

Metrics-based approach [2, 11, 43] evaluates or measures source code elements to take decisions depending on a proper selection of threshold values. This approach cannot detect all the smells, only comparatively easy detectable smells can use the metrics approach. There are traditional software metrics such as LOC, complexity and metrics related to the OOP such as complexity, coupling, cohesion, inheritance, encapsulation, clone, abstraction and polymorphism. This approach usually detects Data Clumps, Refused Bequest, Shotgun Surgery

Large Class, Long Method and Lazy Class. Proper selection of threshold value is the most important factor that affects the accuracy, there is no threshold values standardization to detect the code smells which results in unreliability of this approach. In addition, different code smells require a different set of metrics to be detected.

Search-based approach [2, 46] applies different algorithms and rules to detect code smells. Genetic and heuristic search algorithms are used to extract the rules to detect the smells. This approach learns from the standard and correct code design and best practices then compares the developer coding practices with these standard practices. Machine learning (ML) algorithms are used mostly in the search technique, by choosing a set of predictors for a ML classifier to take a proper decision. ML techniques depend on the quality of a balanced dataset and the quality of the training model [37].

History-based [2, 36] observes how the code elements are changed over time and detects the code smells based on the source code analysis by mining the commit log with the “refactor” word in the commit logs of versioned repositories [38]. Logging refactoring tools automatically record the programmer’s activity in a log file.

Strategy-based [2, 11, 43, 46] defines metrics, logic expression and rules to detect smells or automatically without any standardization between rules. This approach usually detects Divergent Change, Feature Envy and Long Parameter List.

Visualization-based [2, 11, 46] combines the human effort to detect code smells with the help of an automated detection process. This approach is not capable to handle large-scale projects and requires manual effort which can result in human error.

Cooperative-based [46] is a new technique that uses different activities in order to improve the accuracy of the detection process. Applying different parallel algorithms such as genetic algorithms, to reduce the search space and increase the search process speed.

Duplicated code is considered the most popular code smell among all the smells [2, 5, 15], so researchers have investigated clone management and created more detection approaches for clone detection such as Token-Based, Syntax-Based, Hash-Based and Graph-Based Comparisons [15].

Detection approaches use different algorithms, methodologies as:

- ML algorithms: create models from learning rules with input data and cost functions. Some of these ML solutions depend on random forests, decision trees, support vector machines, neural networks and deep learning [43].
- Data Mining: design and source code data are collected in order to improve the software quality by applying data mining algorithms using graphs and trees [43].
- Longest common subsequence: transform the code source and design into tree or graph form to find the longest common subsequences [43].
- Matching and similarity algorithms: search for similar sequences and tokens using multi-pattern

matching algorithms, similarity scoring and suffix tree matching [43].

- Evolutionary optimization: commonly used in the refactoring process, these algorithms such as greedy and genetic algorithms are used to solve optimization problems [43].

A. Detection and Refactoring Tools

Detection and refactoring tools help the developers to apply a better design in less time and high accuracy. A generic detection and refactoring process is illustrated in “Fig. 1”, the input to detection tool can be the source code, intermediate data representation parsed from the source code such as trees, XML and graphs, or metrics extracted from the code. After applying refactoring, a comparison should be done between the code before and after the refactoring process to ensure that this refactoring sequence improves the code quality.

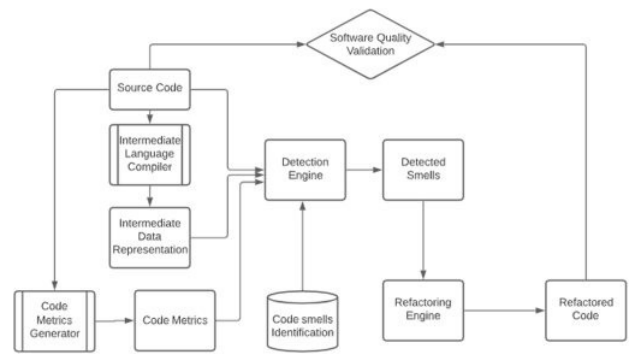


Fig. 1. Generic Detection and Refactoring Process

The most cited tools are presented in Table II with columns tool name, link, creation year, supported IDE, code smells detected and the detection techniques. All the mentioned tools are used for detection only except JDeodorant and TrueRefactor which are used for detection and refactoring. Most of the mentioned tools support Java language more than any other languages as clarified in “Fig. 2”.

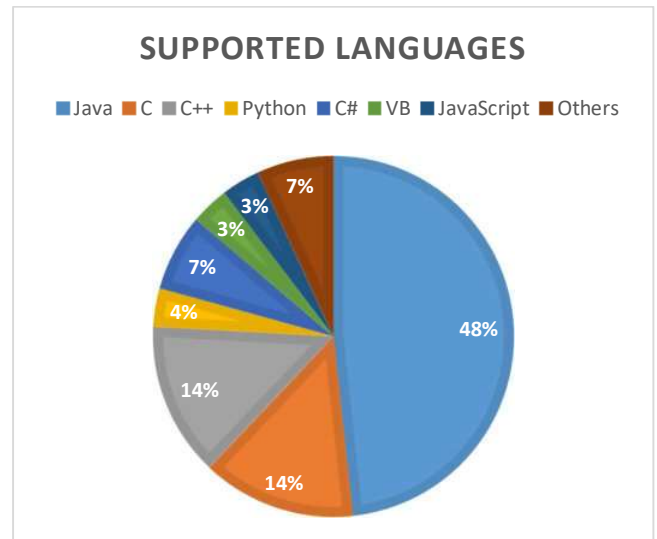


Fig. 2. Supported Languages by Tools

TABLE II. DETECTION AND REFACTORING TOOLS

Tool Name	Link	IDE	Year	Code Smells detected	Detection Technique	References
CCFinder, CCFinderX (an upgraded version of CCFinder)	http://www.ccfinder.net/ https://github.com/radekg1000/ccfinderx	-	2002	Duplicated code	Token-based (Matching and similarity algorithms using tokens, supports Type-1 and Type-2 clones)	[2, 3, 29, 48]
PMD	http://pmd.sourceforge.net/ https://pmd.github.io/ https://github.com/pmd/pmd-eclipse-plugin	Eclipse	2002	Long Parameter List, Duplicated code, Long Method and Large Class	Metrics based (uses NLOC with threshold 1000)	[2, 3, 11, 12, 30, 46]
InCode	https://marketplace.eclipse.org/content/incode-helium	Eclipse	2010	Large Class, Data clumps, long method, Refused Bequest, Shotgun Surgery, Divergent Change, Feature Envy and Duplicated code	Metrics based	[2, 3, 31, 46]
DÉCOR/ DETEX	http://www.ptidej.net/research/designsmells/	Ptidej	2010	Refused Bequest, Large Class, Lazy Class, Long Parameter List, Long Method, Feature envy, Message Chains, Shotgun Surgery, Duplicated Code, Data Class, Divergent change and Speculative Generality	Metrics Based	[2, 3, 32, 43, 46]
inFusion	http://www.intooitus.com/inFusion.html	-	2011	Long Parameter List, Divergent Change, Duplicated code, Large Class, Feature Envy, Long Method, Data Clumps, Refused Bequest and Shotgun Surgery	Metrics based	[2, 3]
iPlasma (open source version of inFusion)	http://loose.cs.upt.ro/index.php?n=Main.IPlasma	Eclipse	2005	Large Class, Feature Envy, Duplicated code, Long Method, Long Parameter List, Refused Bequest and Shotgun Surgery	Metrics based	[2, 3, 11, 12, 46]
Checkstyle	http://checkstyle.sourceforge.net/ https://github.com/checkstyle/checkstyle/	Eclipse	2001	Long method, Duplicated code, Large class and Long Parameter List	Metrics based (uses NLOC with threshold 2000)	[2, 3, 46]
Nicad	https://github.com/bumper-app/nicad	Linux, Solaris, Cygwin, Mac OS X	2011	Duplicated code	Text-based	[2, 33, 43, 49]
JCD	https://www.swag.uwaterloo.ca/jcd/	-	2009	Duplicated code	Search Based (uses pcode matching algorithms)	[2, 34]
CloneDr	http://www.semdesigns.com/Products/Clone/JavaCloneDR.html	Eclipse	-	Duplicated Code	AST-based (detects Type-1 and Type-2 clones)	[2, 35]
HIST	-	-	2011	Large class, Feature envy, Divergent change and shotgun surgery	Search based/ change history information	[2, 5, 36, 43, 44]
StenchBlossom	https://github.com/DeveloperLiberationFront/refactoring-tools	Eclipse	2010	Switch Statement, Data Clumps, Long Method, Message Chain, Large Class, Typecast and Feature Envy	Visualization based and Search based	[2, 11, 12, 28, 46]
JSNOSE	http://github.com/saltlab/JSNose	-	2013	Switch statement, Message chain, Long method, Large object, Coupling, Empty catch, Nested callback, Lazy object, Refused bequest, Long parameter list, Unused/ dead code and Excessive global variables	Metrics based	[43, 46, 47]
JDeodorant	http://www.jdeodorant.com/	Eclipse	2007	Duplicated Code, Feature Envy, God Class, Type/State Checking and Long Method	Search based	[2, 3, 11, 12, 17-26, 43, 44, 46]
TrueRefactor	https://app.assembla.com/wiki/show/truerefactor/TrueRefactor https://app.assembla.com/spaces/truerefactor/git/source	-	2011	Temporary Field, Lazy Class, Long Method, Shotgun Surgery and Large Class	Metrics based	[2, 27]

Tools vary in their accuracy as clarified in [2], PMD works fine in detecting Long Method more than the other smells. Checkstyle and PMD almost detect the same smells, but each tool has a different threshold and accordingly a different result, **PMD has more general performance**. Although **DÉCOR** is the tool that detects the largest number of smells, experiments do not indicate a high accuracy when using this tool. **JDeodorant** and **Eclipse** are the most used for refactoring, PMD and JDeodorant are most used tools for bad smell detection [3].

V. FINDINGS AND CONCLUSION

This literature introduces the limitations and challenges found through our analysis:

1. The majority of tools support Java language rather than any other language, as it is easier to refactor java code more than a language such as C and C++, in addition to that the most popular programming datasets are written in Java.
2. Detection or refactoring tools should be experimented using the same benchmark to indicate a better comparison between tools, and the validation should be done using a standard benchmark dataset.
3. Long Method, Duplicate Code, Long Parameter List and Feature Envy are the most detected code smells by the detection tools.
4. **No tool detects incomplete library class, Primitive Obsession, Inappropriate Intimacy or Comments smells.**
5. Metrics-based approach depends on a threshold value which is not a standard value, resulted in inconsistency.
6. ML algorithms are the most used according to the previous investigations.
7. The performance of ML algorithms depends mainly on the training dataset.
8. Refactoring a code smell could create other new smells, so the relations between code smells and how a code smell refactoring can lead to another smell should be investigated.
9. Not all tools are available, resulting in a huge difficulty to validate their results.

Refactoring Code smells help the developers to improve the code quality in a significant way, this work highlight the identification of code smells, suggested refactoring techniques, define some of the detection strategies and introduces a brief on the most cited refactoring and detection tools. There are many challenges and limitations discussed in this literature survey, indicating that this topic needs lots of research and improvements.

REFERENCES

- [1] Martin Fowler and Kent Beck. 1999. Refactoring: improving the design of existing
- [2] Lacerda, G., Petrillo, F., Pimenta, M., and Guhéneuc, Y.G., 2020. "Code smells and refactoring: A tertiary systematic review of challenges and observations" in Journal of Systems and Software 167, 110610
- [3] Agnihotri, M. and Chug, A., 2020. "A systematic literature survey of software metrics, code smells and refactoring techniques" in Journal of Information Processing Systems 16(4), pp. 915-934
- [4] Wake, W.C. , 2003. Refactoring Workbook. Addison-Wesley .
- [5] Jain, S. and Saha, A., 2019. "An empirical study on research and developmental opportunities in refactoring practices" in Proceedings of the International Conference on Software Engineering and Knowledge Engineering, SEKE 2019-July, pp. 313-318
- [6] Panita M., 2012. "Identifying Refactoring Sequences for Improving Software Maintainability" in 2012 27th IEEE/ACM International Conference on Automated Software Engineering, ASE 2012 – Proceedings pp. 406-409
- [7] Bekefi, B.F., Szabados, K. and Kovacs, A., 2019. "A case study on the effects and limitations of refactoring" in INFORMATICS 2019 - IEEE 15th International Scientific Conference on Informatics, Proceedings 9119321, pp. 213-218
- [8] Taibi, D., Janes, A. and Lenarduzzi V., 2017. "How developers perceive smells in source code: A replicated study" in Information and Software Technology 92, pp. 223-235
- [9] Dos Santos, H.M., Figueiredo, E., Durelli, V.H.S., Souza, M. and Durelli, R.S., 2019. "Cleangame: Gamifying the identification of code smells" in ACM International Conference Proceeding Series pp. 437-446
- [10] Oo, T., Liu, H., Nyirongo and B. ,2018. "Dynamic Ranking of Refactoring Menu Items for Integrated Development Environment" in IEEE Access 6, 8552339, pp. 76025-76035
- [11] Singh, S., Kaur and S., 2018. "A systematic literature review: Refactoring for disclosing code smells in object oriented software" in Ain Shams Engineering Journal 9(4), pp. 2129-2151
- [12] Vidal, S., Berra, I., Zulliani, S., Marcos, C. and Andrés Díaz Pace, J., 2018. "Assessing the refactoring of brain methods" in ACM Transactions on Software Engineering and Methodology 27(1)
- [13] Kerievsky, J. , 2004. Refactoring to Patterns. Addison-Wesley .
- [14] Fowler, M. , Beck, K. , 2019. Refactoring: Improving the Design of Existing Code
- [15] Roy, C.K., Zibran, M.F., and Koschke, R., 2014. "The vision of software clone management: Past, present, and future (Keynote paper)" in 2014 Software Evolution Week - IEEE Conference on Software Maintenance, Reengineering, and Reverse Engineering, CSMR-WCRE 2014 – Proceedings 6747168, pp. 18-33
- [16] Yamashita, A. and Moonen, L., 2013. "Do developers care about code smells? An exploratory survey" in Proceedings - Working Conference on Reverse Engineering, WCRE 06671299, pp. 242-251
- [17] Nikolaos Tsantalis, Davood Mazinanian, and Shahriar Rostami, "Clone Refactoring with Lambda Expressions," 39th International Conference on Software Engineering (ICSE'2017), Buenos Aires, Argentina.
- [18] Nikolaos Tsantalis, Davood Mazinanian, and Giri Panamoottil Krishnan, "Assessing the Refactorability of Software Clones," IEEE Transactions on Software Engineering, vol. 41, no. 11, pp. 1055-1090.
- [19] Giri Panamoottil Krishnan, and Nikolaos Tsantalis, "Unification and Refactoring of Clones," pp. 104-113, IEEE Conference on Software Maintenance, Reengineering and Reverse Engineering (CSMR-WCRE'2014), 2014 Software Evolution Week, Antwerp, Belgium.
- [20] Giri Panamoottil Krishnan, and Nikolaos Tsantalis, 2013. "Refactoring Clones: An Optimization Problem," pp. 360-363, 29th IEEE International Conference on Software Maintenance (ICSM'2013), ERA Track, Eindhoven, The Netherlands, September 22-28.
- [21] Davood Mazinanian, Nikolaos Tsantalis, Raphael Stein, and Zackary Valenta, 2016. "JDeodorant: Clone Refactoring," 38th International Conference on Software Engineering (ICSE'2016), Formal Tool Demonstration Session, Austin, Texas, USA.
- [22] Nikolaos Tsantalis, 2016. "JDeodorant: Clone Refactoring Support beyond IDEs," IEEE Software Blog.
- [23] Nikolaos Tsantalis, and Alexander Chatzigeorgiou, 2011. "Identification of Extract Method Refactoring Opportunities for the Decomposition of Methods," Journal of Systems and Software, vol. 84, no. 10, pp.
- [24] Nikolaos Tsantalis, and Alexander Chatzigeorgiou, 2009. "Identification of Extract Method Refactoring Opportunities," pp. 119-128, 13th European Conference on Software Maintenance and Reengineering (CSMR'2009), Kaiserslautern, Germany.
- [25] Marios Fokaefs, Nikolaos Tsantalis, Eleni Stroulia, and Alexander Chatzigeorgiou, 2012. "Identification and Application of Extract Class Refactorings in Object-Oriented Systems," Journal of Systems and Software, vol. 85, no. 10, pp. 2241-2260.

- [26] Marios Fokaefs, Nikolaos Tsantalis, Alexander Chatzigeorgiou, and Jörg Sander, 2009."Decomposing Object-Oriented Class Modules Using an Agglomerative Clustering Technique," pp. 93-101, 25th IEEE International Conference on Software Maintenance (ICSM'2009), Edmonton, Alberta, Canada.
- [27] Griffith, I., Wahl, S. and Izurieta, C., 2011 ." TrueRefactor: An automated refactoring tool to improve legacy system and application comprehensibility" in Proceedings of the ISCA 24th International Conference on Computer Applications in Industry and Engineering, CAINE 2011 pp. 316-321
- [28] Emerson Murphy-Hill and Andrew P. Black, An interactive ambient visualization for code smells, Proceedings of SOFTVIS '10, USA, October 2010.
- [29] E. H. Vashisht, S. Bharadwaj, and S. Sharma, "Impact of Clone Refactoring on External Quality Attributes of Open Source Softwares," International Journal of Scientific Research in Computer Science, Engineering, and Information Technology, vol. 3, no. 8, pp. 86-94, 2018.
- [30] Pmd.sourceforge.net, PMD, Available at: <http://pmd.sourceforge.net/>
- [31] Radu Marinescu, G. Ganea and Ioana Verebi , 2010."InCode: Continuous Quality Assessment and Improvement" in Software Maintenance and Reengineering (CSMR), 2010 14th European Conference
- [32] Naouel Moha, Yann-Gael Gueheneuc, Laurence Duchien, Anne-Francoise Le Meur, 2010." Decor : a method for the specification and detection of code and design smells", TSE, IEEE CS Press.
- [33] Feng, C., Wang, T., Liu, J., Zhang, Y., Xu, K. and Wang, Y., 2020." NiCad+: Speeding the detecting process of nicad" in Proceedings - 14th IEEE International Conference on Service-Oriented System Engineering, SOSE 2020 9183401, pp. 103-110
- [34] JCD Available at : <https://www.swag.uwaterloo.ca/jcd/>
- [35] Kaurr, H. and Maini, R. ,2020." Function clone removal using refactoring techniques" in Advances in Mathematics: Scientific Journal 9(6), pp. 4001-4013
- [36] Mara, L., Honorato, G., Dantas, F., Garcia, A. and Lucena, C., 2011."Hist-inspect: A tool for history-sensitive detection of code smells" in Proceedings of the 10th International Conference on Aspect-Oriented Software Development Companion, AOSD.11 pp. 65
- [37] Dario Di Nucci, Fabio Palomba, Damian A. Tamburri, Alexander Serebrenik and Andrea De Lucia , 2018." Detecting Code Smells using Machine Learning Techniques: Are We There Yet?" in Conference: 25th IEEE International Conference on Software Analysis, Evolution, and Reengineering
- [38] Palomba F., Bavota G., Di Penta M., Oliveto R., De Lucia A. and Poshyvanyk D. ,2013." Detecting bad smells in source code using change history information" in 2013 28th IEEE/ACM International Conference on Automated Software Engineering, ASE 2013 – Proceedings 6693086, pp. 268-278
- [39] Zhu, J., and Liu, W. ,2020. "A tale of two databases: The use of Web of Science and Scopus in academic papers" in Scientometrics, 123(1), 321-335
- [40] Arcoverde, R., Garcia, A., Figueiredo, E., 2011." Understanding the longevity of code smells preliminary results of an explanatory survey" in Proceedings - International Conference on Software Engineering pp. 33-36
- [41] Drozd, M., Kourie, D.G., Watson, B.W., Boake, A., 2006." Refactoring tools and complementary techniques" in IEEE International Conference on Computer Systems and Applications, 2006 2006, 1618429, pp. 685-688
- [42] Kim, D.J., 2020." An empirical study on the evolution of test smell" in Proceedings - International Conference on Software Engineering 3382176, pp. 149-151
- [43] AbuHassan, A., Alshayeb, M., Ghouti, L., 2020." Software smell detection techniques: A systematic literature review" in Journal of Software: Evolution and Process
- [44] Kaur, A., Dhiman, G., 2019." A review on search-based tools and techniques to identify bad code smells in object-oriented systems " in Advances in Intelligent Systems and Computing 741, pp. 909-921
- [45] Gupta, A., Suri, B., Misra, S., 2017." A systematic literature review: Code bad smells in java source code" in Lecture Notes in Computer Science (including subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics) 10408 LNCS, pp. 665-682
- [46] Rasool, G., Arshad, Z., 2015." A review of code smell mining techniques" in Journal of Software: Evolution and Process 27(11), pp. 867-895
- [47] Amin Milani Fard, Ali Mesbah, 2013." JSNOSE: Detecting javascript code smells" in 13th IEEE Intl. Conf. on Source Code Analysis and Manipulation (SCAM'13), DOI: 10.1109/SCAM.2013.6648192
- [48] Toshihiro Kamiya, Shinji Kusumoto, Katsuro Inoue, 2002." CCFinder: A Multilingual Token-Based Code Clone Detection System for Large Scale Source Code" in IEEE Transactions on Software Engineering 28(7):654- 670, DOI: 10.1109/TSE.2002.1019480
- [49] James R Cordy, Chanchal K. Roy, 2011." The NiCad Clone Detector" in Conference: The 19th IEEE International Conference on Program Comprehension, ICPC 2011, Kingston, ON, Canada, DOI: 10.1109/ICPC.2011.26
- [50] Mantyla, M. , 2003. Bad Smells in Software - A Taxonomy and an Empirical Study. Helsinki University of Technology